

CS3200 Project 2 – Design & Implement a Document Database

By Xiaodi Wang

1. Problem Statement

College students applying for internships often manage dozens of applications across multiple companies and job platforms simultaneously. Without a structured system, it becomes easy to miss deadlines, forget interview times, or lose track of where each application stands. This database tracks the full lifecycle of an internship application from the moment a student submits their materials to a final hiring decision.

In Project 2, this system is redesigned as a MongoDB document database. Instead of storing all entities in fully normalized relational tables, the data model is reorganized into root collections and embedded documents that better represent the application lifecycle.

2. Business Rules

- A student has a profile with their name, university, major, and graduation year.
- A student can submit many applications, but each application belongs to exactly one student.
- Each application targets exactly one job posting at a company.
- A company can have many job postings, but each posting belongs to exactly one company.
- Each job posting has a title, location (remote/onsite/hybrid), and job type (internship/co-op/part-time).
- An application has a submission date, current status (Applied, Online Assessment(OA), Interviewing, Offer, Rejected, Withdrawn), and optional notes.
- A student may go through multiple interview rounds for the same application (e.g., phone screen, technical, behavioral, final).
- Each interview round has a scheduled date, interview type, and outcome (Pending, Passed, Failed).
- A student can store contacts at a company (e.g., recruiters, hiring managers) and associate them with an application.
- A student can tag an application with one or more tags (e.g., "Dream Company", "Referral", "High Priority") for personal organization.
- An application can have at most one offer record, which stores compensation details and an offer deadline.

Project 2 Interpretation:

These business rules remain unchanged in Project 2. However, their implementation changes from a relational schema to a document-oriented MongoDB design. In the MongoDB version, several tightly related entities such as interview rounds, tags, contacts, offer details, and status history are embedded inside application documents.

3. Nouns

- Student
- Company
- JobPosting
- Application
- InterviewRound

- Tag
- ApplicationTag
- Contact
- ApplicationContact
- Offer
- JobBoard
- StatusHistory

In Project 2, not every noun becomes its own MongoDB collection. Some nouns remain root collections, while others become embedded subdocuments within a larger application document.

4. Actions

- Submits
- Targets
- Schedules
- Tags

These actions are reflected in the MongoDB design through document creation, embedded arrays, and updates to nested fields within application documents.

5. MongoDB Logical Redesign

In Project 2, the relational schema from Project 1 is redesigned into a hierarchical MongoDB document model. The new design is centered around the application lifecycle. Instead of storing interview rounds, tags, contacts, offer details, and status history in separate normalized tables, these related objects are embedded within application documents.

This redesign reduces the number of joins required in the relational version and better matches the way internship application data is accessed in practice. Students and companies are kept as separate root collections because they can still be queried independently.

6. Main Collections

The main MongoDB collections used in this project are:

1. students

Stores core student profile information such as name, university, major, and graduation year.

2. companies

Stores company profile information and may include embedded job postings and company contacts.

3. applications

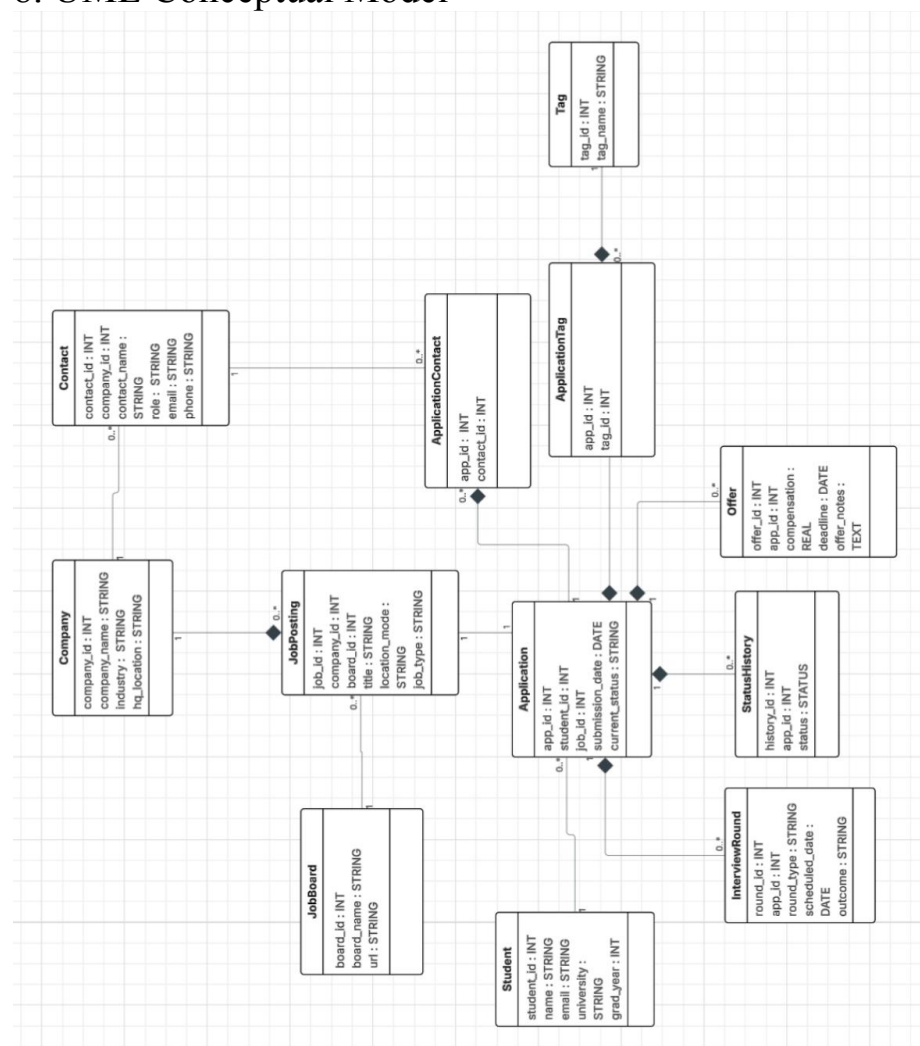
Stores the full lifecycle of an internship application, including embedded student snapshot, company snapshot, job posting snapshot, interview rounds, tags, contacts, offer details, and status history.

7. Embedded Data Design Choices

The applications collection is the central collection in the MongoDB redesign. It embeds interview rounds, tags, contacts, offer details, and status history because these entities are tightly coupled with a single application and are typically accessed together.

The students collection remains separate because a student may have multiple applications. The companies collection also remains separate because a company may have multiple job postings and contacts and may be queried independently across the system.

8. UML Conceptual Model



The UML conceptual model from Project 1 is reused in Project 2 because the business domain and conceptual entities remain the same.

9. MongoDB Logical Model

The MongoDB logical model reorganizes the original relational entities into root collections and embedded subdocuments. The applications collection acts as the main aggregate root for tracking the complete application lifecycle.

10. JSON Document Examples Overview

Representative JSON examples are provided for the students, companies, and applications collections. These examples demonstrate how MongoDB documents store nested and embedded data. In particular, the applications collection includes embedded arrays for interview rounds, tags, contacts, and status history, as well as an embedded offer object when applicable.

11. Data Initialization Overview

The MongoDB database is initialized using JSON files and mongoimport. Separate JSON files are provided for the students, companies, and applications collections. These files can be imported into MongoDB Compass or from the command line using mongoimport.

The following commands can be executed in the terminal to import the data:

```
mongoimport --db internship_tracker_mongo --collection students --file
mongo/students.json --jsonArray
```

```
mongoimport --db internship_tracker_mongo --collection companies --file
mongo/companies.json --jsonArray
```

```
mongoimport --db internship_tracker_mongo --collection applications --file
mongo/applications.json --jsonArray
```

12. MongoDB Query Overview

This project includes at least five MongoDB queries demonstrating the document database design. The queries include:

- a document retrieval query,
- an aggregation query,
- a complex search query using logical operators,
- a count query for a specific student,
- and an update query that modifies a document field.

These queries are stored as separate JavaScript files in the mongo folder.